

#2 Image Convolution Filter Implementation with CUDA

Image Size / Filter	CPU (ms)	GPU Naive (ms)	GPU Constant (ms)	GPU Shared (ms)
256x256 (3x3)	4.7738 ms	0.3913 ms	0.0348 ms	0.0370 ms
512x512 (3x3)	19.2246 ms	0.3911 ms	0.0528 ms	0.0584 ms
1024x1024 (3x3)	118.2041 ms	0.5512 ms	0.1024 ms	0.1269 ms
2048x2048 (3x3)	287.3245 ms	0.7086 ms	0.2867 ms	0.3132 ms
512x512 (5x5)	53.0330 ms	0.4424 ms	0.0656 ms	0.0668 ms
1024x1024 (5x5)	208.1280 ms	0.5795 ms	0.1568 ms	0.1409 ms
2048x2048 (5x5)	919.9516 ms	1.1922 ms	0.5509 ms	0.4453 ms

1. Introduction

Image convolution is a fundamental operation in image processing used for transformations such as blurring, sharpening, and edge detection. This operation applies a mathematical kernel (filter matrix) to each pixel and its neighbors. Because every pixel can be computed independently, image convolution is an ideal candidate for massive parallelization.

The objective of this laboratory work is to design, implement, and analyze four distinct approaches to 2D image convolution: a sequential CPU execution, a baseline (naive) GPU implementation, and two advanced memory-optimized GPU designs utilizing Constant Memory and Dynamic Shared Memory.

2. Implementation Approaches

2.1. CPU Sequential Implementation

The CPU implementation serves as the baseline for correctness and performance comparisons. It uses a straightforward nested 4-loop structure: two outer loops iterate over the image dimensions (**W × H**), while two inner loops slide the filter window (**K_w × K_h**) over the pixel neighborhoods.

Boundary conditions are managed via **Clamping (Padding)**: any attempt to reference an index outside the valid image boundaries is constrained to the nearest edge pixel, preventing memory faults and artifacts.

2.2. GPU Naive Implementation

The naive GPU approach eliminates the two outer loops by mapping each pixel of the output image directly to an individual CUDA thread. Threads are organized into a grid of 2D blocks (**16 × 16**).

Memory Access: Every thread independently fetches both the input pixel values and the filter weights directly from the slow **Global Memory**. This creates a significant memory bandwidth bottleneck, as identical pixel data and filter weights are fetched repeatedly from Global Memory by neighboring threads.

2.3. GPU Constant Memory Optimization

To alleviate the bottleneck caused by fetching filter weights from Global Memory, the convolution kernel matrix is moved to **Constant Memory** (`__constant__ float d_filter[81]`).

Mechanism: Constant memory is physically cached on-chip. Because every thread in a warp reads the exact same filter coefficient at the exact same clock cycle during convolution, the hardware broadcasts this value to all threads simultaneously. This reduces Global Memory traffic substantially for the filter coefficients.

2.4. GPU Shared Memory (Dynamic Tiling) Implementation

The ultimate optimization targets the pixel data itself. Since neighboring threads reuse the same input pixels up to $K_w \times K_h$ times, a **Tiled Convolution** technique using **Dynamic Shared Memory** was designed.

Dynamic Allocation: Instead of hardcoding the shared array dimensions, a clean host utility function `getSharedMemorySize()` computes the memory footprint in bytes dynamically based on block dimensions and current filter radius: *Shared Memory Size* = $(\text{Block}_x + 2R) \times (\text{Block}_y + 2R) \times \text{sizeof}(\text{unsigned char})$

Cooperative Loading & Synchronization: Threads within a block work together to load a data tile including the **Halo (or Apron) pixels** required for boundary calculations. A barrier synchronization token (`__syncthreads()`) is called immediately after loading to guarantee that the entire tile is present in the ultra-fast on-chip Shared Memory before math operations begin.

3. Observed Behavior and Performance Analysis

3.1. CPU vs. GPU Acceleration

The baseline CPU execution suffers severely from sequential processing overhead. At a resolution of **2048 × 2048**, the CPU requires **287.32 ms**. The Naive GPU implementation handles the same task in just **0.7086 ms**, yielding an immediate **x405 speedup**. This confirms the efficiency of CUDA thread scheduling for pixel-dense algorithms.

3.2. The Constant Memory vs. Shared Memory Tradeoff

At Smaller Workloads (256×256 to 2048×2048 with a 3×3 filter):

GPU Constant Memory outperforms Shared Memory (**0.0528 ms** vs **0.0584 ms** at 512×512). This occurs because modern NVIDIA architectures feature incredibly advanced hardware L1/L2 caches that automatically buffer small images effectively. The manual overhead of managing a Shared Memory tile, loading halo pixels through branch loops, and forcing warp-wide stalls via `__syncthreads()` outweighs the memory access benefits when the filter size is minimal ($3 \times 3 = 9$ elements).

At Bigger Workloads (1024×1024 with a 5×5 filter):

As the filter expands to 5×5 (25 elements), the number of redundant global memory accesses scales quadratically. Under this heavy load, **Shared Memory achieves a breakthrough at 0.1409 ms**,

outperforming Constant Memory (**0.1568 ms**). Cooperative tiling proves its architectural dominance when heavy redundant memory reuse causes standard hardware caches to experience capacity evictions.

4. Nsight Systems Theoretical Bottlenecks

Naive Kernel Bottleneck: Bound strictly by *Memory Bandwidth*. High latencies due to uncoalesced global memory requests and continuous cache misses.

Constant Kernel Bottleneck: On large filters ($>9 \times 9$), it hits a *Capacity Limit* (constant memory is strictly capped at 64KB across most hardware platforms).

Shared Memory Kernel Bottleneck: Bound by *Instruction/Synchronization Overhead* on small configurations, which transitions into an optimal execution state once compute intensity increases.

5. Potential Further Optimizations

Vectorized Memory Access (uchar4 / int4): Instead of threads fetching pixel bytes individually (`sizeof(unsigned char)`), memory operations can be vectorized to read 4 or 16 bytes simultaneously using native types like *uchar4*. This maximizes bus width utilization and drastically slashes the absolute count of load instructions.

Asynchronous Stream Overlap (cudaStream_t): By breaking huge images into chunks, memory transfers from Host-to-Device (H2D) and Device-to-Host (D2H) can run concurrently with kernel execution across multiple non-blocking CUDA streams, masking memory copy latencies entirely.

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U
=====
      CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 256x256 | Filter Width: 3

[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using boxBlur3x3!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using boxBlur3x3!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using boxBlur3x3!
=====
      PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      :    4.7738 ms
GPU Naive           :    0.3913 ms (Speedup: x12.20)
GPU Constant Memory :    0.0348 ms (Speedup: x137.11)
GPU Shared Memory   :    0.0370 ms (Speedup: x128.94)
=====
```

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U
=====
      CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 512x512 | Filter Width: 3

[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using boxBlur3x3!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using boxBlur3x3!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using boxBlur3x3!
=====
      PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      :   19.2246 ms
GPU Naive           :    0.3911 ms (Speedup: x49.16)
GPU Constant Memory :    0.0528 ms (Speedup: x364.10)
GPU Shared Memory   :    0.0584 ms (Speedup: x329.01)
=====
```

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U
=====
      CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 1024x1024 | Filter Width: 3

[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using boxBlur3x3!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using boxBlur3x3!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using boxBlur3x3!
=====
      PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      :  118.2041 ms
GPU Naive           :    0.5512 ms (Speedup: x214.44)
GPU Constant Memory :    0.1024 ms (Speedup: x1154.70)
GPU Shared Memory   :    0.1269 ms (Speedup: x931.62)
=====
```

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U:
=====
  CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 2048x2048 | Filter Width: 3

[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using boxBlur3x3!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using boxBlur3x3!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using boxBlur3x3!
=====
  PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      :   287.3245 ms
GPU Naive           :    0.7086 ms (Speedup: x405.48)
GPU Constant Memory :    0.2867 ms (Speedup: x1002.11)
GPU Shared Memory   :    0.3132 ms (Speedup: x917.34)
=====
```

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U:
=====
  CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 512x512 | Filter Width: 5

[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using gaussianBlur5x5!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using gaussianBlur5x5!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using gaussianBlur5x5!
=====
  PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      :    53.0330 ms
GPU Naive           :    0.4424 ms (Speedup: x119.88)
GPU Constant Memory :    0.0656 ms (Speedup: x808.82)
GPU Shared Memory   :    0.0668 ms (Speedup: x793.34)
=====
```

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U:
=====
  CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 1024x1024 | Filter Width: 5

[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using gaussianBlur5x5!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using gaussianBlur5x5!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using gaussianBlur5x5!
=====
  PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      :   208.1280 ms
GPU Naive           :    0.5795 ms (Speedup: x359.14)
GPU Constant Memory :    0.1568 ms (Speedup: x1327.62)
GPU Shared Memory   :    0.1409 ms (Speedup: x1476.84)
=====
```

```
!nvcc my_convolution.cu -o my_convolution
!./my_convolution
```

nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future release (U

```
=====
  CUDA Convolution Lab - Performance Evaluation
=====
Image Size: 2048x2048 | Filter Width: 5
```

```
[-] CPU sequential execution completed.
[-] GPU Naive execution completed.
SUCCESS! CPU and GPU results match using gaussianBlur5x5!
[-] GPU Constant Memory execution completed.
SUCCESS! Constant Memory results match using gaussianBlur5x5!
[-] GPU Shared Memory execution completed.
SUCCESS! Shared Memory results match using gaussianBlur5x5!
```

```
=====
  PERFORMANCE SUMMARY (Time in Milliseconds)
=====
CPU Sequential      : 919.9516 ms
GPU Naive           : 1.1922 ms (Speedup: x771.65)
GPU Constant Memory : 0.5509 ms (Speedup: x1669.87)
GPU Shared Memory   : 0.4453 ms (Speedup: x2066.01)
=====
```